

Stan

Mattias Villani

Introduction

Stan is a probabilistic programming language. This basically means that it is a language for performing statistical inference using a convenient model syntax. Stan is mostly used for Bayesian inference either by

- **simulating** from the posterior distribution using **Hamiltonian Monte Carlo (HMC)** or
- **approximating** the posterior distribution using **variational inference (VI)** methods.

Stan can also be used to obtain maximum likelihood estimates by optimization on a posterior with a uniform prior distribution on the parameters.

The backbone of both the HMC and VI methods is efficient automatic differentiation to automatically compute gradients, which makes it possible for the user to only specify the model and let Stan take care of all the rest.

The user expresses the model and prior in a special Stan syntax, which is then parsed to C++ and eventually compiled to machine code.

We can install R package **rstan** as any other package (use the options below for using all cores and avoiding re-compilation when not needed:

```
#install.packages("rstan")  
library(rstan)
```

Loading required package: StanHeaders

rstan version 2.32.7 (Stan version 2.32.2)

For execution on a local, multicore CPU with excess RAM we recommend calling `options(mc.cores = parallel::detectCores())`.
To avoid recompilation of unchanged Stan programs, we recommend calling `rstan_options(auto_write = TRUE)`
For within-chain threading using ``reduce_sum()`` or ``map_rect()`` Stan functions, change ``threads_per_chain`` option:
`rstan_options(threads_per_chain = 1)`

```
options(mc.cores = parallel::detectCores())
rstan_options(auto_write = TRUE)
```

Stan model syntax

Stan will take care of the posterior sampling, but clearly needs to know about all aspects of the model, data and prior. Stan has a special syntax for defining the statistical model and its parameters. The Stan parser then translates the model into C++ code which is ultimately compiled to fast machine code. The model definition can be written either as a separate Stan file, or as a simple string which is then passed to the parser.

All variables used in the Stan program needs to have defined **types**. Stan needs to know if a variable is integer (whole number), real number, vector or a matrix. It also needs to know about any restrictions on the parameters, so that the HMC algorithm does not propose invalid parameter values. For example,

- a parameter declared as `real<lower=0>` means that the parameters is a continuous positive number (so called floats).
- `int<lower=0, upper=1>` is a integer restricted between 0 and 1, that is, it is a binary variable that can only take on the values 0 and 1.
- The syntax `vector[N]` is a vector with N real numbers.

Note that the default type of the elements in a vector is a real numbers, so that when we do not specify the type the numbers are assumed to be real.

i Data includes also prior hyperparameters

- Stan's use of the term *data* is a little different from the usual terminology: data includes the actual data, but also all prior hyperparameters.

The iid normal model in Stan

Let us consider the simple iid normal model

$$X_1, \dots, X_n | \theta, \sigma^2 \stackrel{\text{iid}}{\sim} N(\theta, \sigma^2)$$

with conjugate prior

$$\theta | \sigma^2 \sim N\left(\mu_0, \frac{\sigma^2}{\kappa_0}\right) \sigma^2 \sim \text{Inv-}\chi^2(\nu_0, \sigma_0^2)$$

The Stan model syntax, here expressed as string named `iidnormal` :

```
iidnormal = '  
  
// data and prior hyperparameters  
data {  
  // data  
  int<lower=0> N;  
  vector[N] y;  
  // prior  
  real mu0;  
  real<lower=0> kappa0;  
  real<lower=0> nu0;  
  real<lower=0> sigma20;  
}  
  
// specify which are the model parameters  
parameters {  
  real theta;  
  real<lower=0> sigma2;  
}  
  
// the model connects parameters to the data  
model {  
  sigma2 ~ scaled_inv_chi_square(nu0, sqrt(sigma20));  
  theta ~ normal(mu0, sqrt(sigma2/kappa0));  
  y ~ normal(theta, sqrt(sigma2));  
}  
'
```

i Stan code is not R code

The above code is not R code. It is Stan's special syntax that will be parsed to something useful for HMC to work on.

Note how the above code has three sections enclosed by angle brackets `{}`:

- **data** - this is where all the data used by model is defined. Recall that also values for the prior hyperparameters is considered data in Stan. We see that the data will be a vector y with N elements. The four hyperparameters in the prior are specified, where we for example see that κ_0 is positive real parameter.
- **parameters** - this specifies the parameters of the model. The HMC sampler will sample from the joint posterior of all the parameters listed here. The iid normal model has only two parameters θ and σ^2 , both a real and σ^2 has to be positive.
- **model** - this is the part that defines the prior and model, using data and parameters defined in the previous two sections. Stan has a large number of pre-defined statistical distributions, including `scaled_inv_chi_square` for the $\text{Inv-}\chi^2$ distribution, and a normal distribution; note that the normal distributions is defined using the standard deviation, not the variance (as in R). Finally, the last line of the model definition section `y ~ normal(theta, sqrt(sigma2))` uses a short-hand syntax where Stan will understand that the elements of the data vector y are *iid* from a normal distribution with mean `theta` and standard deviation `sqrt(sigma2)`. Alternatively, this part can be written as loop:

```
#model {  
#  sigma2 ~ scaled_inv_chi_square(nu0, sqrt(sigma20));  
#  theta ~ normal(mu0, sqrt(sigma2/kappa0));  
#  for (i in 1:N){  
#    y[i] ~ normal(theta, sqrt(sigma2));  
#  }  
#}
```

i Stan comments

Any line starting with `//` is a comment and is ignored by the Stan parser.

Now that the model is defined, let us set up the data and prior hyperparameters. Here we analyze the internet speed data from Chapter 2 in the Bayesian Learning book:

```
# Set up the data
data <- list(N = 5, y = c(15.77, 20.5, 8.26, 14.37, 21.09))

# Set up prior
prior <- list(mu0 = 20, kappa0 = 1, nu0 = 5, sigma20 = 5^2)
```

The code above is plain R code and will run in R.

Now, finally we throw the model definition, data and prior hyperparameters to the `stan` function to sample from the posterior. Note that I pack together the `data` and `prior` list in the `data` argument to the `stan` function. We could have define these together from the start, but it is cleaner to separate data and prior hyperparameters like I have done here. I have four cores on this computer so Stan will run four runs in parallel, each with `iter = 1000` iterations and different initial values. Each such run is referred to as a **chain** in Stan. Have multiple runs is useful for diagnosing convergence: the draws from different runs should give rather similar posteriors (for example histograms). The argument `refresh = 0` makes Stan print less to the screen while running HMC.

```
fit = stan(model_code = iidnormal, data = c(data, prior), iter = 1000, refresh = 0)
```

We can now summarize the posterior, for example by using Stan's built-in summary function:

```
summary(fit, pars = c("theta", "sigma2"))$summary
```

	mean	se_mean	sd	2.5%	25%	50%	75%
theta	16.72395	0.06756018	2.260098	12.31983	15.27716	16.67286	18.18932
sigma2	30.76249	0.67890206	18.755978	12.13421	19.52201	26.28326	35.92057
	97.5%	n_eff	Rhat				
theta	21.30198	1119.1101	1.003438				
sigma2	74.80211	763.2471	1.003074				

We could have just written `summary(fit)`, which would return a summary for all parameters and for all four chains/runs. By only printing out the summary element (hence the `$summary` at the end) we get a more compact summary that merges the draws from all four chains.

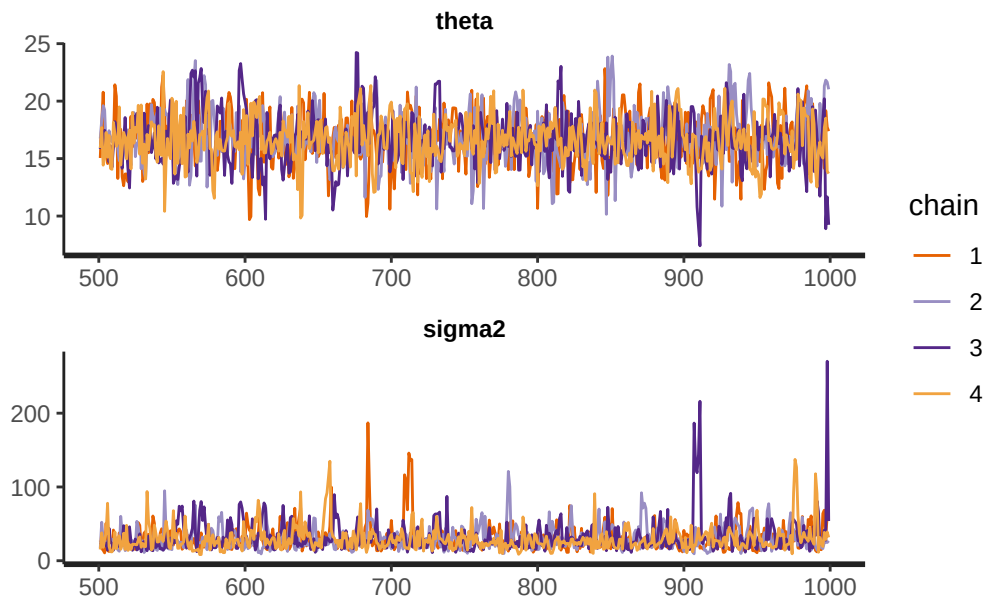
The output has several columns:

- `mean` is the posterior mean

- **se_mean** is the Monte Carlo standard error of the posterior mean estimate in the first column. This can be used, via the central limit theorem, to state that a 95% frequentist confidence interval for the posterior mean estimate is $16.67 \pm 1.96 \cdot 0.076 = (16.521, 16.819)$ (the numbers may not exactly be the ones in the table above due to randomness across runs). This standard error and the confidence interval has nothing to do with the posterior standard deviation. The **se_mean** is purely a numerical standard deviation telling us how precisely HMC can estimate the posterior mean.
- **sd** - this is the posterior standard deviation.
- the columns named 2.5%, 25% etc are posterior quantiles.
- **n_eff** is the number of *effective* draws in each chain. It is defined as **iter**/IF, where IF is the inefficiency factor. We asked for **iter**=1000 draws, but due to the autocorrelation of the draws from HMC we only got the equivalent of **n_eff** *iid* draws. For this simple model, HMC is very efficient: the number of effective draws **n_eff** is almost the same as the nominal number of draws **iter**; we can therefore deduce that $IF \approx 1$, and the draws are close to the ideal *iid* case.
- **Rhat** in the final column is a measure convergence that compares the mean from the different runs using a so called ANOVA analysis. It looks at all parameters jointly (i.e. technically Multivariate ANOVA). The ideal case is **Rhat**=1, and the rule-of-thumb is that **Rhat** should be below X for the sampler to have converged.

We can use Stan's **traceplot** function to plot the MCMC trajectories for all parameters and for each chain (the different colors of the trajectories).

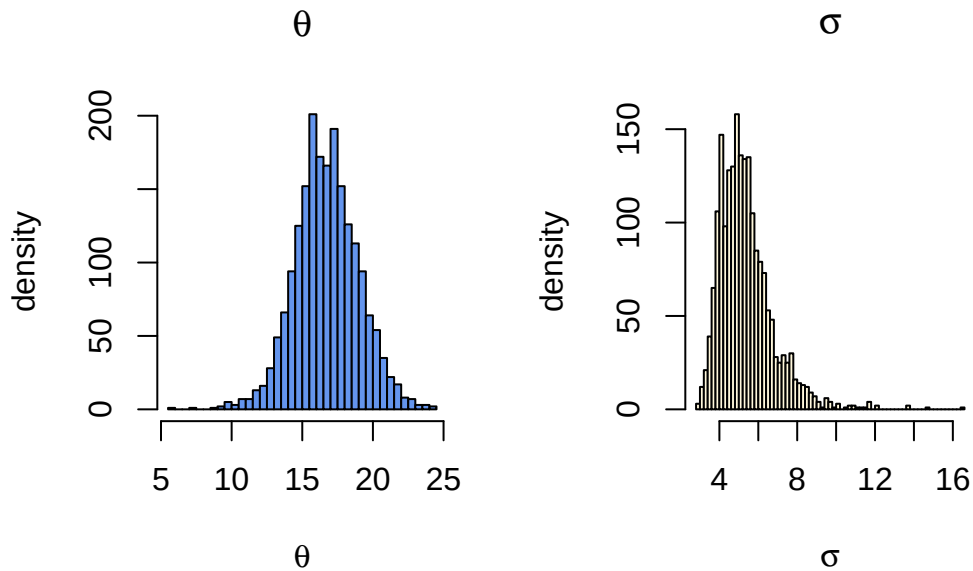
```
# Plot trajectories
traceplot(fit, pars = c("theta", "sigma2"), nrow = 2)
```



The sampler is mixing rapidly, there are no clearly trending patterns and the chains seem to have similar behavior and visiting similar regions over the iterations; convergence seems fine, as we have already seen with the `n_eff` and `Rhat` metrics.

The draws in the `fit` object can be extracted with the `extract` function, and the plotted using your favorite plotting functions in R. Note how I take the square root of the `sigma2` draws and plot the histogram of the posterior standard deviation σ , which is more interpretable.

```
# extract draws
postDraws = extract(fit, pars = c("theta", "sigma2"), permuted = TRUE)
par(mfrow = c(1,2))
hist(postDraws$theta, 50, freq = TRUE, col = "cornflowerblue",
      xlab = expression(theta), ylab = "density", main = expression(theta))
hist(sqrt(postDraws$sigma2), 50, freq = TRUE, col = "cornsilk",
      xlab = expression(sigma), ylab = "density", main = expression(sigma))
```



Generated quantities and Predictions

There is a fourth section that can be added to a Stan program: `generated quantities`. Here we can for example:

- compute functions of the parameters in the `parameter` section
- generate draws from the predictive distribution

Here is an example using the iid normal model where we directly compute the standard deviation and the coefficient of variation $\frac{\theta}{\sigma}$ and generate draws from the predictive distribution:

```
iidnormal = '
// data and prior hyperparameters
data {
  // data
  int<lower=0> N;
  vector[N] y;
  // prior
  real mu0;
  real<lower=0> kappa0;
  real<lower=0> nu0;
  real<lower=0> sigma20;
}
```

```
// specify which are the model parameters
parameters {
  real theta;
  real<lower=0> sigma2;
}

// the model connects parameters to the data
model {
  sigma2 ~ scaled_inv_chi_square(nu0, sqrt(sigma20));
  theta ~ normal(mu0, sqrt(sigma2/kappa0));
  y ~ normal(theta, sqrt(sigma2));
}

generated quantities {
  real<lower=0> sigma = sqrt(sigma2);
  real cv = theta/sigma;
  real yPred = normal_rng(theta, sigma);  // one posterior predictive draw
}
'
```

Running the sampler again with the new model definition (which requires a new compilation):

```
fit = stan(model_code = iidnormal, data = c(data, prior), iter = 1000, refresh = 0)
```

Model comparison using leave-one-out predictive cross-validation

The `loo` package, made by some of Stan developers, can be used to assess the predictive performance of a model using leave-of-out cross-validation. This means that the model is fitted to all data points, except one data which is left as test data to evaluate the model prediction. This is then repeated so that all data points are used exactly once to evaluate the predictions. Since we are doing Bayesian inference, and we obtain a predictive distribution, we often would like to evaluate the predictive density, not just a point prediction using root mean squared error, or some other commonly used metric for point predictions. A useful so called scoring rule is to evaluate the log predictive density of the test data